Deadlocks

Deadlock differs from starvation and ordinary waiting. Necessary conditions, resource graphs, safe-state analysis, detection, and recovery require different information and impose different costs.

Prerequisites. Process states, Mutexes and semaphores, Resource-allocation graphs, Basic vector comparisons.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Learning objectives

- Distinguish necessary from sufficient deadlock conditions
- Prove how a global lock order excludes circular waiting
- Apply the Banker's safety test using complete resource vectors

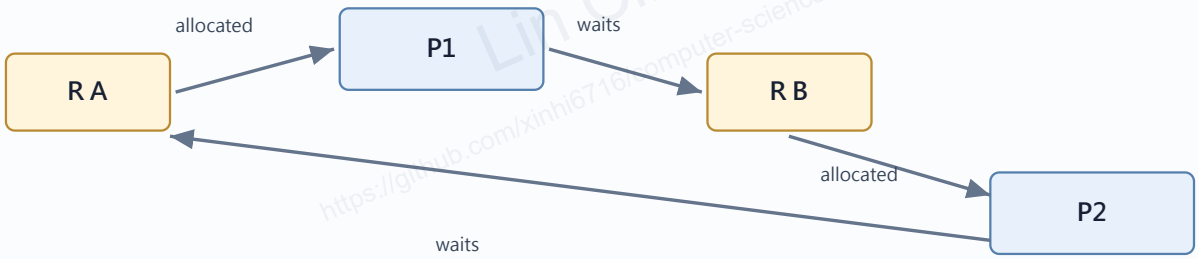
Central question. When can resource waiting become permanent, and which prevention or recovery strategy is justified by available information?

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.1. System model

- A resource type has a defined number of instances. Processes request, use, and release instances; an unsatisfied request may block.
- In a resource-allocation graph, process-to-resource edges represent requests and resource-instance-to-process edges represent allocations. The graph records specified ownership and waiting, not every possible future action.

Single-instance cycle: A → P1 → B → P2 → A



Two single-instance owners wait for each other and cannot be preempted: deadlocked.

7.1. Model, ordering, and boundary cases

Model assumptions. Instance counts are fixed in this model. Ownership and requests are observed consistently.

Invariant and ordering. Each exclusive instance has at most one owner. Every edge corresponds to a real request or allocation.

Counterexample or boundary. An unrecorded available instance can allow progress despite an apparent cycle.

Worked check. R1 has two instances held by P1 and P2. P1 waits for R2; P2 requests another R1. P3 holds R2 but can finish and release it. P1 then finishes and releases R1, allowing P2 to proceed. The partial waiting picture was insufficient to prove deadlock.

Reasoning prompt. Which additional instance or releaser can break an apparent cycle?

Resource state and concrete deadlocks

A resource type groups interchangeable instances. Two printers with incompatible location/function requirements belong to different types. Tables record type totals, free/allocated instances, owners, and each outstanding requester/type. A deadlocked set has every member waiting for an event only another waiting member can cause.

With three CD-RW drives, P1/P2/P3 each holds one and requests one more. Available=0 and none can finish, so all three deadlock. With one printer and one DVD, P1 holds DVD and requests printer; P2 holds printer and requests DVD. Neither can release, so their distinct resource types still deadlock.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.2. Deadlock characterization

- The classical resource deadlock requires mutual exclusion, hold and wait, no preemption, and circular wait to hold together. These conditions guide prevention; a cycle alone is sufficient only under the relevant single-instance model.
- With multiple instances, another runnable owner can release an instance and break the apparent dependency. Analyze whether any legitimate completion sequence remains.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.2. Model, ordering, and boundary cases

Model assumptions. Resources are exclusive, held while other requests wait, and not forcibly reclaimable. The system's permitted release behavior is specified.

Invariant and ordering. A deadlocked set cannot progress through its outstanding requests under those rules. Mere delay or lack of current CPU execution does not establish this.

Counterexample or boundary. Let R1 have one instance held by P1, and R2 two held by P2 and P3. P1 waits for R2 and P2 for R1. If P3 finishes, it frees R2 and breaks the cycle. This argument cannot be applied to two single-instance mutexes by inventing another owner.

Worked check. For single mutexes A and B, A is allocated to P1, P1 waits for B, B is allocated to P2, and P2 waits for A. If neither owner can proceed or release and neither lock is preemptible, the four-node cycle is deadlocked.

Reasoning prompt. What future-release information does a snapshot omit?

Exact graph criteria and a multiple-instance counterexample

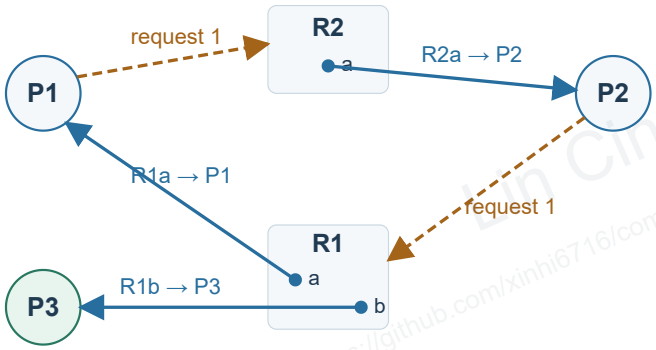
Mutual exclusion makes at least one resource exclusive. Hold-and-wait keeps an allocation while another request waits. No preemption forbids forced reclamation. Circular wait gives a dependency chain returning to its start. Circular wait already implies hold-and-wait; the four conditions are not logically independent.

A process node is a circle, resource type a box, instances dots. Request Pi→Rj becomes assignment Rj→Pi when granted; release removes the assignment. No graph cycle implies no deadlock. When every resource on a cycle has one instance, the cycle is necessary and sufficient for deadlock. With multiple instances it is only a necessary warning.

7.2. Why can a cycle still make progress?

R1 has instances R1a/R1b; R2 has R2a. P3 needs nothing more. Each process finishes and returns all holdings after its current request is granted. Available=(0,0).

Initial state: Total=(2,1), Available=(0,0)



Can finish; no more requests

Read arrow direction

Solid: instance → owner

Dashed: waiter → resource type

R1 contains two distinct instances.

Cycle: P1 → R2 → P2 → R1 → P1

P3 holds the other R1 instance.

Let P3 finish to unlock progress.

P3 can release R1b even while P1 and P2 wait on the cycle.

7.2. Finish P3, then unblock P2 and P1

Completion order P3 → P2 → P1; vector order (R1,R2)

Event	R1a owner	R1b owner	R2a owner	Available
Initial: P1 and P2 wait	P1	P3	P2	(0, 0)
P3 finishes; returns R1b	P1	free	P2	(1, 0)
Grant R1b to P2	P1	P2	P2	(0, 0)
P2 finishes; returns R1b, R2a	P1	free	free	(1, 1)
Grant R2a to P1	P1	free	P1	(1, 0)
P1 finishes; returns R1a, R2a	free	free	free	(2, 1)

A grant decreases Available; an actual completion returns holdings and increases it. The completion order P3→P2→P1 alone omits the grants to P2 and P1. The figure includes both intermediate deductions.

7.2. Detection's Work is a simulation, not live inventory

Vectors use order (R1,R2). Current outstanding requests and existing allocations give:

Process	Allocation	Request	Work before	Work after simulated completion
P3	(1,0)	(0,0)	(0,0)	(1,0)
P2	(0,1)	(1,0)	(1,0)	(1,1)
P1	(1,0)	(0,1)	(1,1)	(2,1)

Check Request≤Work before each step, then set Work←Work+Allocation. This combines granting Request with later returning Allocation+Request, for a net increase of Allocation only. Work may include returns by processes simulated as finished. **The simulation does not execute them; initial Available remains (0,0).**

This test uses **current outstanding Request**, not the Banker's maximum remaining Need. Section 7.6 gives the complete detection algorithm.

7.2. Change only P3's next action to obtain deadlock

Keep every allocation and the requests of P1 and P2. Now require P3 to obtain one R2 before it can finish, adding P3→R2.

Initial state	P3 Request	Eligible at Work=(0,0)	Result
Original	(0,0)	P3	Completion order P3→P2→P1
Modified	(0,1)	None	P1, P2, P3 are deadlocked

P1 and P3 now both wait for R2a, held by P2. Both instances of R1 needed by P2 are held by those waiting processes. Every instance is trapped within the waiting set, with no owner able to return one first. This assumes no preemption, cancellation, extra instances, or early release before completion. Merely delaying P3's CPU service does not change resource feasibility in the original case; scheduling starvation and resource deadlock are different questions.

7.2. A single-instance lock cycle

Thread A acquires `first` then `second`; B uses the reverse order.

Time	Action	Holdings and waits
t0	A acquires first	A holds first
t1	B acquires second	A holds first; B holds second
t2	A requests second	A holds first and waits for B
t3	B requests first	B holds second and waits for A

Both locks have one instance. Neither owner can proceed or release, and no external holder can help. This interleaving deadlocks. If A finishes before t1, that schedule avoids the cycle, but one successful stress schedule cannot exclude the bad schedule.

7.3. Methods for handling deadlocks

- Prevention structurally removes a necessary condition. Avoidance examines whether a proposed allocation preserves safety using declared future demands. Detection permits dangerous allocations, then identifies deadlock and recovers.
- A general-purpose system may leave some deadlocks to applications because global prevention or detection is costly. That choice does not remove application lock-order obligations.

7.3. Model, ordering, and boundary cases

Model assumptions. Maximum claims, runtime costs, and the consequences of aborting work are known to the degree the selected policy requires.

Invariant and ordering. Every accepted allocation follows that policy's guarantee. Avoidance cannot rely on demands absent from its model.

Counterexample or boundary. A safety test using an understated maximum claim can approve allocations that are unsafe for the real demands.

Worked check. With one R1 and one R2, prevention forbids reverse-order acquisition. Avoidance checks declared claim edges before granting the dangerous request. Detection permits the state and later scans the wait-for cycle to choose a victim. These mechanisms act at different times with different information.

Reasoning prompt. Which dominates the design choice, missing future information or the cost of recovering a failure?

7.4. Deadlock prevention

- Exclusive devices may inherently require mutual exclusion. Remove hold-and-wait by acquiring everything together or releasing before another request. Preemption works only when state can be safely saved and restored. A global acquisition order removes circular waiting.
- All-at-once allocation can waste capacity. Preemption can require rollback. Ordering is often practical for software locks but must include dynamic objects and indirect calls.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.4.1. Lock-order proof

- If a thread may request a new lock only above every lock it already holds, each dependency follows an increasing rank. A cycle would require rank(A)<...<rank(A), a contradiction.
- Callbacks, lock upgrades, condition-wait reacquisition, and exceptional paths must obey the same rule. Checking only the normal path is incomplete.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.4. Model, ordering, and boundary cases

Model assumptions. All acquisition paths, including callbacks, upgrades, and errors, obey the model. Reclaimed resources have a consistent restoration protocol.

Invariant and ordering. Strictly increasing acquisition ranks cannot form a directed cycle. State must be consistent before a resource becomes available to another participant.

Counterexample or boundary. A normal A-then-B path and a repair B-then-A path can deadlock despite the usual path's ordering.

Worked check. Order A before B. T1 can hold A and wait for B; a T2 holding B cannot acquire A. The checker rejects that second request before waiting, reporting expected A<B and actual B→A. Rejection does not release B: the caller must explicitly restore a consistent state, release B, and retry in A-then-B order. Ignoring the error while retaining B forever can still leave T1 waiting indefinitely.

Reasoning prompt. How can dynamic resources receive a stable total order without depending on mutable names?

Policy information and cost

Policy	Information	Guarantee	Cost
Prevention	Restricted request protocol	Removes a necessary condition	Underutilization, refused work, rollback
Avoidance	Trustworthy declared maxima	Every grant preserves a safe sequence	Claims and runtime safety checks
Detection/recovery	Current ownership/request snapshot	Finds and repairs allowed deadlocks	Delay, scanning, lost victim work
Ignore	No general deadlock protocol	No general guarantee	Application timeout or restart

For one R1 and one R2, prevention orders F(R1)=1<F(R2)=2; a holder of R2 cannot request R1. Avoidance uses prior claim edges and refuses a grant that would create a cycle. Detection grants first and scans the wait-for cycle later. Their decision times differ.

Policy information and cost (continued)

Sharing read-only files removes exclusion only for those resources. Acquiring everything initially removes hold-and-wait but can reserve an idle printer until the final stage. Releasing everything before a new request may destroy uncommitted state. Preemption is useful only for restorable resources; it can reclaim a blocked process's allocations and restart when all needed resources become available. Strict ordering removes circular wait.

With injective F:R→N, a cycle would require F(R0)<F(R1)<…<F(Rn)<F(R0), hence F(R0)<F(R0), a contradiction. Sort dynamic account locks by stable ID before acquiring them; from/to parameter order can reverse across transactions. Callback, upgrade, wait reacquisition, and error paths participate in the same proof.

7.5. Deadlock avoidance

- The Banker model uses Available, Allocation, Max, and Need=Max-Allocation. Claims are upper bounds, each no greater than total resources, and requests stay within those claims. A single-instance claim graph is a related but different representation.
- A state is safe if some order can satisfy each remaining Need from current Work, then recover the completed process's Allocation. Safe does not mean nobody waits. Unsafe means that such a worst-case guarantee is absent, not that deadlock has already occurred.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.5. Model, ordering, and boundary cases

Model assumptions. Resource totals are fixed, snapshots are consistent, and participants eventually finish and return allocations once their needs are met.

Invariant and ordering. Initialize Work=Available. Repeatedly choose unfinished i with Need_i<=Work componentwise, set Work=Work+Allocation_i, and mark i finished. Safety requires all processes to finish in the reduction.

Counterexample or boundary. A process in an unsafe state might use less than its maximum or release resources early. That possible future does not establish the Banker's worst-case safety guarantee.

Worked check. Starting Work=(2,1,0), choose a process whose complete Need fits. Add only its existing Allocation on simulated completion; do not add imaginary resources or double-count the temporary grant, which is returned at completion.

Reasoning prompt. Why does deadlock imply unsafety in this model while unsafety does not imply current deadlock?

Claim edges and the complete Banker algorithm

For single instances, a claim Pi→Rj means Pi may request Rj later. A request changes claim to request, then trial assignment. Grant only if the resulting graph remains acyclic. Claims must be known before start, or added only while that process holds no assignments. Missing claims invalidate the guarantee. An adjacency-matrix cycle method costs $O(n^2)$.

For n processes and m resource types, Need=Max−Allocation and comparisons are componentwise. Legal inputs have nonnegative integers and $0 \le \text{Allocation} \le \text{Max} \le \text{Total}$.

```
Work = Available
Finish = [false] * n
while there is unfinished i with Need[i] <= Work:
    Work += Allocation[i]
    Finish[i] = true
safe iff all Finish
```


Claim edges and the complete Banker algorithm (continued)

Once Need_i≤Work, lending Need_i and receiving Allocation_i+Need_i has net effect Work−Need_i+(Allocation_i+Need_i)=Work+Allocation_i. It neither manufactures Need nor double-counts the temporary loan. Assume participants finish and return everything once their declared remaining needs are met.

The greedy choice needs no backtracking. If a safe order exists and a different currently feasible Pi is chosen first, move Pi to the front and keep all other processes in their original order. Returning Pi's nonnegative allocation only increases resources faced by every remaining step. If the algorithm stops unfinished despite a supposed safe order, take the first unfinished Pj in that order. Every predecessor has already been removed by the algorithm; current Work contains their allocations and possibly more. Thus Need_j≤Work, contradicting the absence of a candidate. Completion provides a safe-order certificate; stopping unfinished proves unsafety within this model.

Resource-request transaction

Validate an m-component nonnegative integer Request_i. If any component exceeds Need_i, reject as exceeding the claim. Otherwise if any component exceeds Available, wait. Negating componentwise $\leq$ means some component is greater, not all components.

Trial Available−=Request, Allocation_i+=Request, Need_i−=Request. Run safety. Commit only if safe; otherwise restore all three arrays and wait. The update is transactional. Typical safety complexity is $O(mn^2)$, at most n scans of n rows with m-component tests.

Counterexample: Total=(2,1), Allocation=(1,0), Need=(1,1), Available=(1,1) is legal. Request=(2,0) exceeds Need and Available in the first component. It must error before trial. Requiring both components to be greater would wrongly allow Available=(−1,1).

Banker T0 and every Work state

Total=(10,5,7), Available=(3,3,2).

Process	Allocation	Max	Need
P0	(0,1,0)	(7,5,3)	(7,4,3)
P1	(2,0,0)	(3,2,2)	(1,2,2)
P2	(3,0,2)	(9,0,2)	(6,0,0)
P3	(2,1,1)	(2,2,2)	(0,1,1)
P4	(0,0,2)	(4,3,3)	(4,3,1)

Each next Need fits. Work proceeds (3,3,2)→P1 (5,3,2)→P3 (7,4,3)→P4 (7,4,5)→P0 (7,5,5)→P2 (10,5,7). Thus ⟨P1, P3, P4, P0, P2⟩ is safe.

P1 requests (1,0,2), within Need(1,2,2) and Available. Trial gives Available=(2,3,0), Allocation1=(3,0,2), Need1=(0,2,0). The same order yields Work (2,3,0)→(5,3,2)→(7,4,3)→(7,4,5)→(7,5,5)→(10,5,7), so grant.

Banker T0 and every Work state (continued)

After that grant, P0 requests (0,2,0). Although available, its trial leaves Work=(2,1,0). No remaining Need fits, including P0's updated Need=(7,2,3), so rollback and wait. Unsafe means the declared worst-case completion guarantee is missing. Voluntary early release or smaller actual demands can still avoid a real deadlock.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Same free resources, different safety decisions

Use resource types A and B, Total=(4,3), and initial state S with Available=(1,1). Every vector is ordered A then B. The two trials below start independently from S.

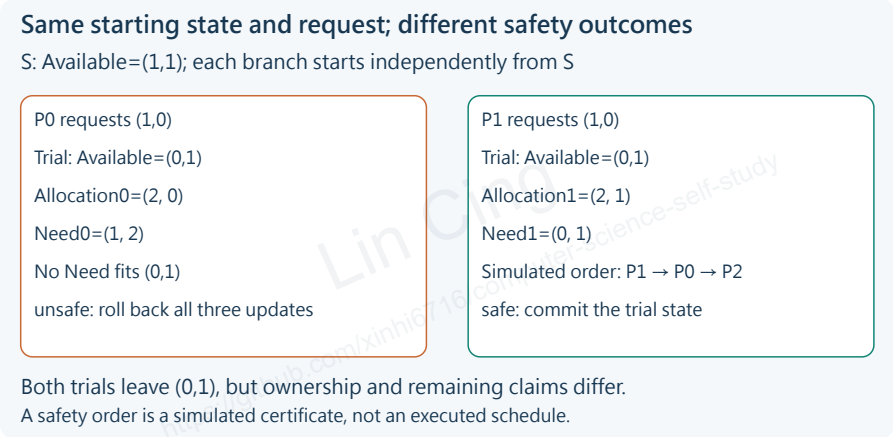
Process	Allocation	Max	Need=Max−Allocation
P0	(1,0)	(3,2)	(2,2)
P1	(1,1)	(2,2)	(1,1)
P2	(1,1)	(3,2)	(2,1)

Allocated totals (3,2), so adding Available recovers Total. Each row satisfies Allocation+Need=Max and Max≤Total. Resources are nonpreemptible and returned together at completion. A process finishes in finite time once its actual demand is supplied.

S is safe: P1 fits Work=(1,1), then returns its allocation to give (2,2). P0 gives (3,2), then P2 gives (4,3). This is a simulated safety certificate, not a required CPU schedule.

Who receives the request changes the result

Each trial starts from S and requests (1,0). Both requests fit the requester's Need and Available. The second trial does not follow the first.



Availability alone is insufficient. Giving A to P1 leaves it needing only one B to finish. Giving the same A to P0 leaves nobody able to finish against its maximum remaining claim.

Reject P0: identify every failing component

Call P0's trial state U. Available=(0,1); only P0 changes, to Allocation0=(2,0), Need0=(1,2). Initialize Work=(0,1) and all Finish entries false.

Candidate	Trial Need	Failing component
P0	(1,2)	A: 1>0; B: 2>1
P1	(1,1)	A: 1>0
P2	(2,1)	A: 2>0

No candidate exists, so safety fails immediately. Restore S: add (1,0) to Available, subtract it from Allocation0, and add it to Need0. Leave other rows and Max unchanged.

Restoring only Available double-counts A. Available+ΣAllocation would be (5,3), exceeding Total=(4,3). Other requests must not observe a partial transaction. Protect check/commit with a lock, or validate the snapshot version at commit and retry after interference.

Grant P1: safety does not execute processes

Restart at S. Trial P1's (1,0) gives Available=(0,1), Allocation1=(2,1), Need1=(0,1). This table updates only simulated Work and Finish.

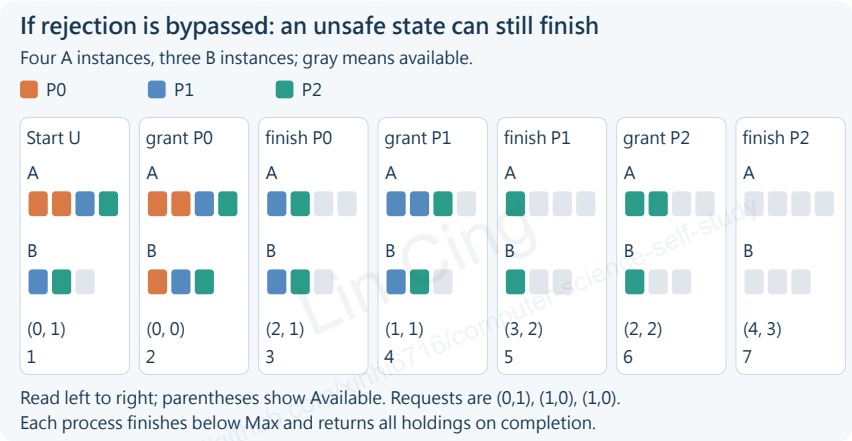
Simulated finish	Need≤old Work	Return Allocation	New Work	Finish: P0/P1/P2
Initial	—	—	(0,1)	F/F/F
P1	(0,1)≤(0,1)	(2,1)	(2,2)	F/T/F
P0	(2,2)≤(2,2)	(1,0)	(3,2)	T/T/F
P2	(2,1)≤(3,2)	(1,1)	(4,3)	T/T/T

Commit the **trial state**, whose Available remains (0,1). Do not copy final Work=(4,3) into Available or treat simulated Finish as actual termination. Real instances return only after actual completion.

Work adds Allocation because granting Need and later returning Allocation+Need has net change −Need+(Allocation+Need)=Allocation. Need creates no resources.

Unsafe does not mean deadlocked: an actual completion path

Suppose P0's rejected trial U is committed anyway. Max bounds demand; it need not be exhausted.



Requests (0,1), (1,0), (1,0) fit the claims in U. This is **one** successful future, not a worst-case guarantee.

The same U also permits deadlock

At U, suppose P0/P1/P2 instead request their entire remaining Need: (1,2), (1,1), (2,1). Each request respects Max, but Available=(0,1) satisfies none. All block while retaining A. With no cancellation, preemption, or early return, this is deadlock.

The two futures share Allocation, Max and Available; their actual requests differ. Safety uses maximum remaining Need to certify a conservative completion order. Detection uses current outstanding Request. Do not silently turn a possible future Need into a present Request.

An unsafe state has no safe sequence, yet can have a successful actual execution. Safe sequence is defined against maximum claims. An observed completion order need not be a safe sequence in that technical sense.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.6. Deadlock detection

- A consistent wait-for graph detects cycles for exclusive single-instance resources. With multiple instances, reduce a snapshot using current outstanding Request rather than maximum future Need.
- Detection frequency trades checking cost against time spent blocked. The result describes the snapshot rather than every future execution.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.6. Model, ordering, and boundary cases

Model assumptions. Available, current requests, and ownership form one consistent snapshot.

Invariant and ordering. Repeatedly satisfy a current Request_i<=Work and return Allocation_i to Work. Interpret the remaining holders according to the detection algorithm's initialization rules, including processes holding no resources.

Counterexample or boundary. Observing ownership before a release but availability after an unrelated update can misclassify the state.

Worked check. With Work=(1,0,0), one request (1,0,0) can finish. If its allocation includes a second-type resource, returning that resource can permit the next request (0,1,0).

Reasoning prompt. What snapshot or locking protocol makes the reduction meaningful?

Detection reduction and recovery states

For single instances, replace Pi→Rq→Pj by Pi→Pj. A cycle is deadlock exactly under the exclusive single-instance assumptions. Any multiple-instance resource requires the allocation/request analysis.

```
Work = Available
Finish[i] = (Allocation[i] == zero_vector)
while there is i with not Finish[i] and Request[i] <= Work:
    Work += Allocation[i]
    Finish[i] = true
remaining false holders form the detected blocked set
```

Detection uses current Request, not maximum Need. It optimistically assumes a satisfiable process completes without additional requests. New requests require a later snapshot. A consistent snapshot and $O(mn^2)$ reduction are required.

Detection reduction and recovery states (continued)

Available=(0,0,0). The complete base snapshot is:

Process	Allocation	Request
P0	(0,1,0)	(0,0,0)
P1	(2,0,0)	(2,0,2)
P2	(3,0,3)	(0,0,0)
P3	(2,1,1)	(1,0,0)
P4	(0,0,2)	(0,0,2)

Work transitions P0→(0,1,0), P2→(3,1,3), P3→(5,2,4), P1→(7,2,4), P4→(7,2,6). All Finish become true, so no deadlock in this snapshot.

Detection reduction and recovery states (continued)

Change only Request2 to (0,0,1). Simulating P0 gives Work=(0,1,0); no other Request fits, so Finish=(true, false, false, false, false) and the detected set is {P1, P2, P3, P4}. Simulation does not actually complete P0 or change Available. If P0 has not finished, aborting P1 and safely reclaiming Allocation1=(2,0,0) changes actual Available from (0,0,0) to (2,0,0).

Restart reduction with Work=Available, excluding the terminated P1:

Simulated completion	Current Request	Work after returning Allocation
Initial; P1 terminated	—	(2,0,0)
P0	(0,0,0)	(2,1,0)
P3	(1,0,0)	(4,2,1)
P2	(0,0,1)	(7,2,4)
P4	(0,0,2)	(7,2,6)

Detection reduction and recovery states (continued)

Check Request≤Work before each row and add only the existing Allocation. This certifies that survivors can finish; actual Available rises only when they really return resources. Starting at actual Available=(2,1,0) is valid only if P0 has already completed outside the simulation.

Frequent scans reduce detection delay and can identify the latest requester, but cost more. Periodic scans can see several cycles together. A snapshot does not guarantee every future state.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.7. Recovery from deadlock

- Recovery can abort all or selected participants or reclaim suitable resources. Victim choice considers work lost, allocations, priorities, and repeated victimization.
- Taking a lock away does not restore partially modified data. Recovery may need transactions, checkpoints, or a restart of the affected subsystem.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.7. Model, ordering, and boundary cases

Model assumptions. The recovery unit and all resource ownership can be reconstructed. If rollback is required, a usable boundary exists.

Invariant and ordering. Recovery exposes a consistent shared version. Victim selection must satisfy any required starvation restriction.

Counterexample or boundary. Releasing a lock over a half-updated tree lets the next owner observe broken links.

Worked check. Track checkpoint positions, rollback costs, and prior victim counts. Prefer low-cost victims subject to a rule that prevents unlimited repeated selection.

Reasoning prompt. Which evidence distinguishes a committed success from work discarded during recovery?

Recovery unit and victim fairness

Abort all deadlocked processes to break every cycle at the cost of all unfinished work, or abort one, reclaim its resources, rerun detection, and stop once survivors can progress. Victim policy considers priority, interactivity, executed/remaining time, held/needed resources, reclamation difficulty, and how many aborts are needed.

Files and devices can retain partial external effects. Recovery needs undo, journals, checkpoints, or device reset, not merely unlocked mutexes. Preemption must specify victim, consistent rollback point, and protection against repeated victimization.

T2 has checkpoint cost 2 and rollback count 0; T4 cost 3/count 0. Choose T2, roll back and release 2 resources. Rerun detection; if T4 now completes, stop aborting. Increment T2's rollback count. A cost such as execution_i+k×rollback_count_i penalizes repeated selection, but a formal finite bound also needs workload assumptions. An explicit policy can exclude a process after at most r consecutive victimizations, for finite r.

Recovery unit and victim fairness (continued)

A single-instance P1/P2 wait cycle is deadlock. Separately, a ready P3 selected zero times while P4 completes five admissions is evidence of possible starvation; only an unbounded execution or policy argument proves no finite bound. No fabricated wait cycle is needed. Ordering can remove deadlock while a CPU policy still starves the lock holder.

Observation	Conclusion	Additional evidence
Four conditions	Deadlock possible	No remaining completion transition
Single-instance graph no cycle	No deadlock in snapshot	Validated graph
Single-instance cycle	Deadlock	All cycle resources truly single-instance
Multi-instance cycle	Deadlock possible	Availability/reduction
Unsafe	No worst-case avoidance guarantee	Actual current requests
One long wait while others finish	Possible starvation	Admission policy and waiting bound

7.8. Summary

- The four conditions guide prevention. Allocation graphs, avoidance, and detection answer different questions under different information assumptions.
- Deadlock prevents a dependent set from progressing. Starvation can leave one participant waiting while others continue.

7.8. Model, ordering, and boundary cases

Model assumptions. Deadlock, unsafety, cycles, and starvation are distinct classifications.

Invariant and ordering. Allocation and recovery conserve resources and specify the scheduling assumptions needed for progress.

Counterexample or boundary. An ordered locking protocol can exclude cycles while a lock owner is still indefinitely denied CPU time by another policy.

Worked check. Apply a necessary-condition check, a resource graph, and a per-participant progress analysis separately.

Reasoning prompt. Give a progressing execution in which one participant nevertheless starves.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter conclusions

- Necessary conditions alone do not establish every deadlock claim
- Avoidance depends on valid maximum claims
- Detection, recovery, and data consistency must be designed together

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exercises and sources

- the separate exercises PDF.
- the separate solutions PDF.

Reference: A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 9th edition, Wiley, 2013. Chapter 7, printed pages 315–339; resource-allocation graph criteria in Section 7.2, detection in Section 7.6.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study